

Received 9 October 2024; revised 14 October 2024; accepted 14 October 2024; Date of publication 17 October, 2024; date of current version 7 November, 2024.

Digital Object Identifier 10.1109/IEEEDATA.2024.3482286

Meta: *Toward a Unified, Multimodal Dataset for Network Intrusion Detection Systems*

SYED WALI ¹, YASIR ALI FARRUKH ¹, IRFAN KHAN ¹,
AND NATHANIEL D. BASTIAN ²

¹Clean and Resilient Energy System Lab (CARES), Department of Electrical & Computer Engineering, Texas A&M University, College Station, TX 77843 USA

²Army Cyber Institute, Department of Electrical Engineering & Computer Science, United States Military Academy, West Point, NY 10996 USA

CORRESPONDING AUTHOR: Syed Wali (e-mail: syedwali@tamu.edu).

This work was supported in part by the U.S. Military Academy (USMA) under Cooperative Agreement W911NF-22-2-0081.

ABSTRACT The lack of standardization across publicly available network intrusion detection datasets presents a significant challenge in developing generalizable machine learning-based models. These existing datasets often exhibit inconsistencies in feature sets and typically focus only on flow-level data, overlooking critical elements such as payload information and time-window-based contextual features. Such limitations make it difficult to detect sophisticated, time-sensitive attacks that rely heavily on both payload analysis and temporal patterns in network behavior. To address these challenges, we propose a unified multimodal dataset that integrates flow, payload, and contextual features from several renowned datasets. Our methodology implements a three-stage pipeline that processes raw packet capture (PCAP) files, extracts detailed metadata, and synchronizes the flow and payload data with time-based contextual features to ensure a comprehensive and enriched dataset. This approach allows for extensive cross-dataset validation, enabling the development of more robust and adaptable machine learning models for network intrusion detection. By providing a unified feature space and incorporating payload and contextual data, the dataset enhances the ability to detect a wider range of network threats with greater precision.

IEEE SOCIETY/COUNCIL Communications Society (COMSOC)

DATA TYPE/LOCATION CSV, Network traffic (flow and payload content); n/a

DATA DOI/PID 10.21227/d8at-gb29

INDEX TERMS Machine learning, multimodal dataset, network intrusion detection systems (NIDSs), payload, security.

BACKGROUND

Network intrusion detection systems (NIDSs) are critical tools in identifying and mitigating malicious activities within network traffic, ensuring the protection of critical systems from unauthorized access and disruptions [1]. Unlike traditional rule-based systems, modern NIDSs leverage machine learning to detect a broader spectrum of threats by learning from complex patterns in large datasets. These machine learning models are trained to recognize various types of attacks based on network data, which can be categorized into two primary forms: packet payloads and flow-level

information [2]. This shift from static, rule-based detection to dynamic, data-driven approaches allows NIDSs to more effectively detect both known and emerging threats in real time [3].

Packet payloads contain the actual data transmitted between network entities, while flow-level information provides aggregated statistics about the communication between endpoints, such as data transfer volumes, connection durations, and interaction frequencies. Flow-based approaches are particularly effective for detecting large-scale anomalies, such as denial of service (DoS) attacks, which generate

abnormally high-traffic volumes. However, flow-based methods often struggle to detect more subtle, content-based attacks that are embedded within individual packets. On the other hand, packet-based NIDSs focus on examining the payloads to identify specific exploits, such as structured query language (SQL) injection and buffer overflow attacks, where the malicious behavior lies within the content itself [4], [5]. Although each approach has strengths, there is increasing recognition that combining both flow and packet-based analysis can enhance detection accuracy across a broader range of network intrusions [6].

To train machine learning models for NIDS, researchers have developed benchmark datasets that simulate network behavior under both normal and attack conditions. These datasets, often captured in packet capture (PCAP) format, provide labeled examples of benign and malicious network flows [7]. The datasets are generated in controlled environments, using network testbeds to simulate traffic between multiple nodes while introducing various attack scenarios. This data are then processed into flow-level information for use in machine learning models. While these publicly available datasets have been instrumental in advancing NIDS research, they also exhibit several limitations that hinder the generalization and scalability of machine learning-based detection models [8], [9].

First, there is a lack of consistency in the feature sets across datasets, as different institutions use varying tools and parameter settings to generate flow-level data. This results in feature spaces that differ significantly between datasets, preventing effective cross-dataset validation and limiting the ability to train machine learning models on a comprehensive set of attack classes. While some efforts have been made to standardize the feature space, they have typically provided labeled flow-based features, restricting studies that require corresponding payload data [8]. Additionally, these efforts do not provide any tool to extend the dataset further, limiting research to the predefined datasets. Therefore, a unified feature space that includes both payload and flow modalities is essential for enabling cross-dataset learning and improving the generalization capabilities of NIDS models.

Second, most publicly available datasets provide only flow-level features, neglecting the payload information that is critical for detecting attacks where the malicious activity is embedded in the data content rather than the flow itself. To address this gap, we previously developed a tool, Payload Byte [10], which labels individual packet payloads as either benign or malicious. However, practical NIDSs require both flow and payload data to overcome the limitations of each. This requires linking packet payloads with their corresponding flows to enable more comprehensive intrusion detection.

Last, current datasets lack time-window-based features that are crucial for detecting attacks that evolve over time, such as distributed denial of service (DDoS) attacks. These attacks may change their behavior gradually, and without contextual or historical information about the targeted net-

work's state over a given time period, it becomes challenging to accurately detect such threats. For example, analyzing how many internet control message protocol (ICMP) requests were received by a server within a specific time frame can significantly reduce false positives and improve the detection of ICMP-based DDoS attacks.

To address these limitations, we have developed a tool and a multimodal, unified intrusion detection dataset that standardizes data from multiple widely used sources. This approach enables the integration of different network data modalities, including both flow-level and payload-based features, creating a comprehensive dataset for more effective intrusion detection. Our tool not only standardizes feature extraction across datasets but also introduces additional contextual features, such as time-window-based information, providing a historical perspective on network activity. The dataset currently incorporates four major datasets—CIC-IDS 2017 [11], CIC-DDoS 2019 [12], UNSW-NB15 [13], and CIC-IoT 2023 [14]—and can be further extended through our GitHub tool, which allows the processing of any raw PCAP files into this standardized feature space. This unified dataset offers a comprehensive view of network intrusions, enabling cross-dataset validation and the development of more robust NIDS models.

PROPOSED METHODOLOGY

To create a unified multimodal intrusion detection system (UM-IDS) dataset that includes network flow data, associated packet payloads, and additional contextual features, we developed a comprehensive three-stage processing pipeline. Each stage is designed to systematically transform raw PCAP files into a labeled, structured dataset ready for analysis and model training. The pipeline consists of the following stages.

Stage I: PCAP Processing

As shown in Fig. 1, the first stage involves the parallel processing of raw PCAP files, which are collected from multiple datasets. To efficiently handle the large volume of data, each PCAP file is assigned to a worker for independent processing, significantly enhancing the speed and scalability of the system.

During this stage, special consideration is given to handling large files. If a PCAP file exceeds 2 GB, it is automatically split into smaller comma-separated values (CSV) files. This mechanism ensures efficient memory usage and facilitates the smooth processing of large-scale network traces without overloading the system, supporting the scalability of the pipeline when dealing with massive datasets.

This efficient file management is complemented by the use of NFStream [15], a versatile network traffic analysis tool that serves as the core component of the PCAP processing stage. To enhance its functionality, a customized plugin was developed to extract detailed flow and packet-level features. The plugin captures standard flow metrics such as bidirectional packet counts, source-to-destination packet

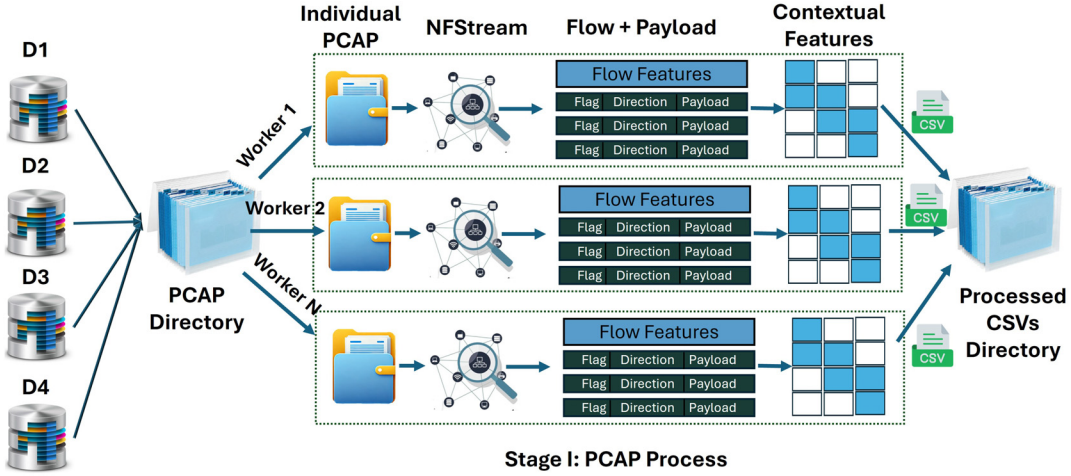


FIG. 1. Stage I: PCAP processing pipeline—this figure illustrates the parallel processing of raw PCAP files sourced from multiple datasets using NFSstream. Each worker independently extracts flow features, such as packet flags, payload content, and contextual time-based features. The extracted data are saved as CSV files in the processed CSV directory, preparing the dataset for the subsequent metadata extraction and labeling stages.

counts, and protocol-specific information (e.g., transmission control protocol (TCP), user datagram protocol (UDP), and ICMP counts), as well as more granular data such as packet flags (synchronize (SYN), acknowledgment (ACK), finish (FIN), reset (RST), and push (PSH)) and the payload content of each packet. This detailed information provides a deeper layer of analysis, offering insights into sophisticated threats such as SQL injections and data exfiltration attempts, which are often missed in traditional datasets focused solely on flow-level data.

Additionally, the pipeline generates contextual features using a sliding time-window approach. These features capture important historical and temporal information, such as the number of SYN requests, ACK responses, and TCP/UDP packets exchanged within a specified time frame. By aggregating features over source-destination pairs and individual destinations, the system can track variations in packet size, flow duration, and port activity over time. This provides critical insights into time-sensitive attacks, such as advanced persistent threats (APTs), where timing patterns and packet sequences play a crucial role.

After feature extraction, the processed data are stored in the processed CSVs directory, ensuring that the outputs are structured and ready for the next stage of data labeling. Each worker within the system produces its own CSV file, allowing for efficient parallel processing of large-scale network traces. This design enables the system to effectively manage large datasets, ensuring that contextual, flow, and payload-level features are comprehensively captured and organized for downstream analysis.

Stage II: Metadata Extraction

In the second stage, metadata are extracted from the pre-labeled CSV files provided by the dataset developers. Each worker processes these CSV files individually, converting the timestamps into a standardized Unix time format. This

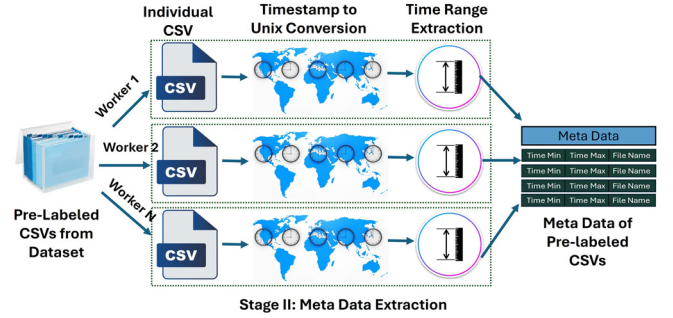


FIG. 2. Stage II: Metadata extraction—this figure showcases the metadata extraction process where individual CSV files are processed to normalize timestamps and extract time ranges. The metadata, which includes the minimum and maximum timestamps along with the file name, is generated for each processed file. This metadata are essential for aligning the processed network flows with pre-labeled data in the final stage.

timestamp normalization ensures consistency across datasets, which may have been recorded in different time zones, allowing for seamless integration in the subsequent processing stages.

Once the timestamps are normalized, the system extracts the minimum and maximum timestamps for each CSV file, which are then stored as metadata. This metadata are essential for aligning the pre-labeled CSV files with the processed network flow data in the final stage. To handle large datasets efficiently, the metadata extraction process is also parallelized for optimal performance. As shown in Fig. 2, the metadata consists of the time range and corresponding file name, which are used to identify the relevant data for labeling.

Stage III: Unified Multimodal Dataset Labeling

The final stage involves labeling the processed CSV files to create the unified multimodal dataset. Using the metadata extracted in stage II, the system identifies the relevant

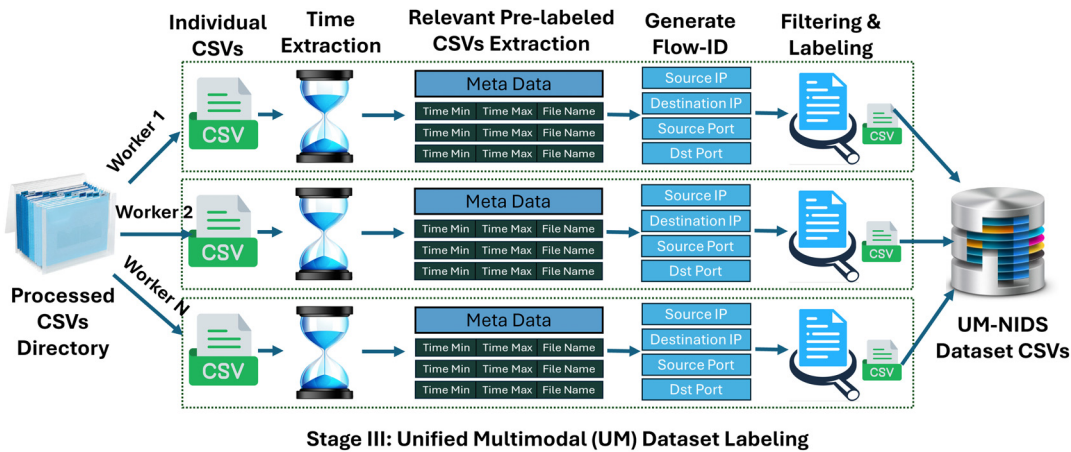


FIG. 3. Stage III: Unified multimodal dataset labeling—this figure presents the final stage of the pipeline, where processed CSVs are matched with pre-labeled datasets based on the extracted metadata and generated flow IDs. Once the matching is complete, the system applies the appropriate labels, classifying the network traffic as malicious or benign, and produces a fully labeled unified multimodal dataset (UM-NIDS) for further analysis and model training.

prelabeled CSV files that correspond to the processed data within the same time range. This ensures that the flows from the PCAP data are accurately matched with their corresponding labels.

After identifying the relevant pre-labeled CSVs from the metadata, each worker extracts flow-level information, such as source internet protocol (IP), destination IP, source port, and destination port, to generate a unique flow identifier (ID). This flow ID is then used to match flows between the processed CSVs and the pre-labeled datasets. Once the matching is complete, the system assigns the appropriate labels, classifying the traffic as either malicious or benign based on both time alignment and flow ID.

This labeling process completes the creation of the unified multimodal (UM)-NIDS dataset by merging the flow, payload, and contextual features from stage I with the labels from the pre-labeled datasets. The final labeled dataset is saved in a structured format and is ready for analysis and model training. Fig. 3 illustrates the entire labeling process, from time extraction and flow ID generation to filtering and labeling the dataset.

By integrating these three stages, our methodology ensures the generation of a comprehensive, unified multimodal dataset that incorporates flow and payload data, contextual features, and accurate labels. This structured dataset is ideal for building robust machine learning models for network intrusion detection.

APPLIED ANALYSIS

In this section, we explain the steps taken to prepare and evaluate the undersampled version of the UM-NIDS dataset. We also discuss how we evaluated the impact of contextual features and payload-based analysis, facilitating cross-dataset validation and demonstrating the benefits of creating a unified dataset.

Data Management and Sampling

Managing large-scale datasets can be challenging, especially when combining multiple sources with a diverse range of attack classes. To address this, we implemented a sampling strategy that capped the number of instances per class at 5000, creating a more manageable dataset while maintaining the essential characteristics of each attack class. Table I presents the data distribution for the undersampled version of UM-NIDS, where we applied a window size of 350 and used the default tool settings. This undersampled version of UM-NIDS is available on IEEE Dataport. Additionally, our GitHub tool allows users to modify the window size and other parameters, offering flexibility to adjust thresholds according to specific research needs, making the dataset adaptable for various applications.

Feature Comparison and Cross-Dataset Validation

To evaluate the impact of contextual features, we compared the performance of machine learning classifiers with and without these features across all four datasets. By using the same classifier for both scenarios, we were able to clearly demonstrate the performance improvements gained from the inclusion of contextual features. This direct comparison highlights the value of integrating these additional features, showing how they enhance the classifier's ability to detect network intrusions more effectively.

Additionally, we conducted cross-dataset validation by training a classifier with term frequency-inverse document frequency (TF-IDF) on payload-specific attacks from the CIC-IoT 2023 dataset and evaluated its performance on both the test set and payload-based attacks in the CIC-IDS 2017 dataset. This validation highlights the value of payload-based analysis, made possible by our unified dataset, for detecting such attacks. We then compared this method to a flow-based analysis using the same classifier, revealing the

TABLE I. Undersampled UM-NIDS Data Distribution

Category	Number of Samples	Source
DNS_Spoofing	5000	CIC_IoT_2023
Benign	5000	CIC_IoT_2023
DoS_GoldenEye	5000	CIC_IDS_2017
MITM-ArpSpoofing	5000	CIC_IoT_2023
Mirai-greeth_flood	5000	CIC_IoT_2023
Mirai-greip_flood	5000	CIC_IoT_2023
Mirai-udpplain	5000	CIC_IoT_2023
Recon-HostDiscovery	5000	CIC_IoT_2023
Recon-PortScan	5000	CIC_IoT_2023
Exploits	5000	UNSW-NB15
DDoS-UDP_Flood	5000	CIC_IoT_2023
Fuzzers	5000	UNSW-NB15
DrDoS_DNS	5000	CIC_DDoS
DrDoS_LDAP	5000	CIC_DDoS
DrDoS_MSSQL	5000	CIC_DDoS
DrDoS_NTP	5000	CIC_DDoS
DrDoS_NetBIOS	5000	CIC_DDoS
DrDoS_SNMP	5000	CIC_DDoS
DrDoS_SSDP	5000	CIC_DDoS
DDoS-UDP_Fragmentation	5000	CIC_IoT_2023
TFTP	5000	CIC_DDoS
DDoS-TCP_Flood	5000	CIC_IoT_2023
DDoS-SlowLoris	5000	CIC_IoT_2023
DDoS-SYN_Flood	5000	CIC_IoT_2023
DDoS-RSTFINFlood	5000	CIC_IoT_2023
DDoS-PSHACK_Flood	5000	CIC_IoT_2023
DDoS-ICMP_Fragmentation	5000	CIC_IoT_2023
DDoS-ICMP_Flood	5000	CIC_IoT_2023
DDoS-ACK_Fragmentation	5000	CIC_IoT_2023
DoS_Hulk	5000	CIC_IDS_2017
DDoS-SynonymousIP_Flood	5000	CIC_IoT_2023
FTP-Patator	3974	CIC_IDS_2017
Generic	3861	UNSW-NB15
SSH-Patator	2978	CIC_IDS_2017
SqlInjection	2830	CIC_IoT_2023
Shellcode	1433	UNSW-NB15
Bot	1228	CIC_IDS_2017
BrowserHijacking	972	CIC_IoT_2023
CommandInjection	275	CIC_IoT_2023
Backdoor_Malware	244	CIC_IoT_2023
XSS	222	CIC_IoT_2023
Worms	156	UNSW-NB15
Infiltration	122	CIC_IDS_2017
Heartbleed	70	CIC_IDS_2017
DoS_Slowhttptest	29	CIC_IDS_2017

limitations of relying solely on flow data in identifying specific attack types.

To further validate the advantages of a multimodal approach, we compared the performance of a flow-based NIDS model with a model that integrates both flow and payload data. While the flow-based model focuses solely on packet

flow information, the multimodal model combines this data with payload features, providing a detailed network status. This comparison highlighted that multimodal NIDS can better detect attacks that exploit specific payload characteristics, such as web-based attacks and certain reconnaissance techniques, thereby enhancing the overall detection capabilities of the system.

Performance Evaluation on Undersampled Dataset

We further demonstrated the versatility of the UM-NIDS dataset by training several machine learning classifiers on the undersampled version of the dataset. The performance of these classifiers was evaluated across different attack classes, illustrating the ease of use and adaptability of our dataset for various types of network attacks. These classifiers were assessed using standard evaluation metrics defined in the subsequent section.

Evaluation Metrics

To evaluate the performance of the machine learning-based NIDS in all experiments, we used three widely accepted evaluation metrics: precision, recall, and F1-score. These metrics are defined as follows.

- 1) Precision: It measures how many of the instances predicted as positive (malicious traffic) are actually correct. It is calculated as the ratio of true positives (correctly identified malicious traffic) to the total predicted positives (all instances predicted as malicious)

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}. \quad (1)$$

- 2) Recall: It measures how many of the actual positive instances (malicious traffic) are correctly identified by the model. It is calculated as the ratio of true positives to the total actual positives (all instances that are truly malicious)

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}. \quad (2)$$

- 3) F1-Score: It provides a balance between precision and recall, especially useful when there is an imbalance between classes. It is the harmonic mean of precision and recall and ensures that both are equally weighted in the performance evaluation

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (3)$$

These metrics offer a comprehensive understanding of how well the model identifies malicious traffic while minimizing false positives and false negatives.

DISCUSSION OF RESULTS

In this section, we present the results of our experiments, where we evaluated the performance of various machine learning classifiers across different datasets, with and without contextual features, as well as the impact of flow-based versus payload-based analysis.

TABLE II. Performance Comparison With and Without Contextual Features

Features	Evaluation Metric	CIC-IDS 2017	CIC-IoT 2023	UNSW NB 15	CIC-DDoS 2019
Without	Precision	0.86	0.85	0.75	0.56
Contextual	Recall	0.79	0.81	0.67	0.47
Features	F1-Score	0.80	0.83	0.70	0.43
With	Precision	0.99	0.96	0.82	0.76
Contextual	Recall	0.93	0.96	0.72	0.74
Features	F1-Score	0.93	0.96	0.75	0.74

Contextual Feature Impact

In the first experiment, we assessed the performance of a random forest classifier on several datasets, both with and without the inclusion of contextual features. The process began with data cleaning, where we removed features that could bias the classifier, such as source and destination IP addresses, port numbers, and starting timestamps. We also excluded any rows containing erroneous data. Afterward, the dataset was split into a 70:30 ratio for training and testing.

Once the dataset was prepared, we trained the classifier using the training set having contextual features and evaluated its performance on the test set. To compare, we then removed the contextual features from the training set and repeated the evaluation on the same test set. This direct comparison allowed us to clearly demonstrate the improvements in performance gained from including contextual features.

As shown in Table II, the results demonstrate a substantial improvement in classifier performance when contextual features are included. For instance, on the CIC-IDS 2017 dataset, the F1-score increased from 0.80 without contextual information to 0.93 with it. Similar trends were observed in the CIC-IoT 2023 dataset, where the F1-score rose from 0.83 to 0.96. Even in the more challenging CIC-DDoS 2019 dataset, contextual features helped the classifier achieve an F1-score of 0.74, compared to just 0.43 without them. These results underscore the importance of contextual information in improving the accuracy and reliability of machine learning models in detecting network intrusions.

Results of Cross-Dataset Validation

In the second experiment, we evaluated the efficacy of payload- and flow-based analysis on two payload-specific attacks, SQL injection, and cross-site scripting (XSS), from the CIC-IoT 2023 dataset. To prepare the dataset, we first converted the `udps.payload_data` feature, which contains hex-based arrays in string format, into ASCII format. Next, we cleaned the payload data by removing all non-ASCII characters and eliminating rows containing erroneous or corrupt data.

After the cleaning process, we split the dataset into a 70:30 ratio for training and testing. Next, we applied term frequency-inverse document frequency (TF-IDF) to the cleaned payload data, converting the textual information

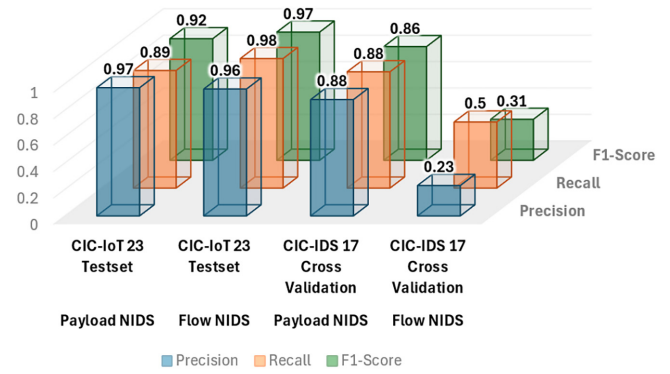


FIG. 4. Performance comparison between flow- and payload-based NIDS for the CIC-IoT 2023 and CIC-IDS 2017 datasets. The figure highlights the significant difference in detection accuracy, precision, recall, and F1-score between test sets and cross-validation sets, showing that payload-based analysis is critical for identifying sophisticated attacks such as SQL injection, which are not easily detected using flow-based features alone.

into numerical features. These features were then used to train a random forest-based machine learning classifier. The trained payload-based NIDS model was evaluated on the test set to assess its detection capabilities. Additionally, to evaluate its robustness, we cross validated the model using payload-specific attacks from the CIC-IDS 2017 dataset, ensuring the model's adaptability across different datasets.

For comparison, we also trained the same random forest classifier using only flow-based features, excluding the payload data. This approach allowed for a direct comparison between the two methods, demonstrating the performance differences between payload- and flow-based analysis in detecting network intrusions.

As shown in Fig. 4, the results indicate that while flow-based analysis performs well on the test set, it struggles to detect payload-specific attacks in the cross-validation set. The classifier achieved an impressive F1-score of 0.97 on the CIC-IoT 2023 test set, but this dropped significantly to 0.31 on the CIC-IDS 2017 cross-validation set. In contrast, the payload-based approach achieved a much higher F1-score of 0.86 on the cross-validation set, demonstrating its superiority in detecting payload-specific attacks. This clearly indicates that payload-based analysis is essential for identifying sophisticated attacks such as SQL Injection and XSS, which cannot be detected through flow-based features alone. Our dataset and tool provide both payload and flow-based information, enabling the development of more comprehensive and effective classifiers for intrusion detection.

Comparison of Multimodal Features

There are several methods that can be utilized to combine flow and payload features into a unified model, such as heterogeneous graphs and ensemble models. In this experiment, we used the TF-IDF method to combine these modalities. The goal was to compare the performance of a NIDS model using only flow-based features with a model that integrates

TABLE III. Performance Comparison of Flow-Based and Multimodal NIDS

NIDS Type	Class	Precision	Recall	F1-Score
Flow-based NIDS	benign	0.87	0.92	0.89
	brute	0.95	0.93	0.94
	ddos	0.99	0.97	0.98
	dos	0.91	0.92	0.92
	mirai	1.00	1.00	1.00
	recon	0.56	0.62	0.59
	spoofing	0.92	0.88	0.90
	webbased	0.88	0.93	0.90
Multimodal NIDS	benign	0.85	0.91	0.88
	brute	0.95	0.93	0.94
	ddos	0.99	0.97	0.98
	dos	0.91	0.95	0.93
	mirai	1.00	1.00	1.00
	recon	0.62	0.67	0.65
	spoofing	0.92	0.87	0.90
	webbased	0.89	0.94	0.91

Note: The bold entries indicate the classifier (Flow-based or Multimodal NIDS) that performs better for each specific class based on the F1-Score. If both approaches achieve the same F1-Score, no entry is bolded, indicating a tie.

both flow and payload-based (multimodal) features. We conducted the experiment using our standardized version of the CIC-IoT 2023 dataset.

First, the dataset was split into training and testing sets with an 80:20 ratio. The flow-based NIDS model was trained using a random forest classifier on the training set, and its performance was evaluated on the test set.

For the multimodal NIDS model, we processed the payload data of each flow. The payload was converted from hex to ASCII, and non-ASCII characters were removed. Next, we trained the TF-IDF vectorizer on the training set to convert the cleaned payload data into numerical features. Using this trained TF-IDF model, we identified the top words for each class from the payload data. We refined these words by removing those that were only numbers or not found in WordNet. These refined words were used to build a vocabulary for the TF-IDF vectorizer, and the resulting output features were combined with the flow-based features.

The combined feature set from both the flow and payload data was then used to train a random forest classifier. The multimodal NIDS model was evaluated on the same test set as the flow-based model. The results, shown in Table III, demonstrate that adding payload-based features improved the classifier's performance, particularly for attacks that rely on payload data,

Results on Undersampled UM-NIDS Dataset

In the final experiment, we evaluated the performance of several machine learning classifiers on the undersampled

TABLE IV. Performance of Machine Learning Models on UM-NIDS Dataset

Machine Learning Model	Precision	Recall	F1 Score
Random Forest	0.92	0.92	0.92
Decision Tree	0.90	0.90	0.90
Multilayer Perceptron	0.70	0.62	0.61
K-Nearest Neighbors	0.84	0.82	0.82

version of the UM-NIDS dataset, which includes contextual features. As summarized in Table IV, random forest and decision tree models performed the best, with both achieving an F1-score of 0.92 and 0.90, respectively. K-nearest neighbors followed with an F1-score of 0.82, while the multilayer perceptron underperformed compared to other classifiers, with an F1-score of 0.61. These results show that even on an undersampled dataset, models incorporating contextual information can achieve high-detection performance. For a more detailed breakdown of class-specific results, please refer to our GitHub repository.

CONCLUSION

In this article, we developed a comprehensive three-stage processing pipeline for creating a UM-NIDS dataset. This dataset integrates network flow data, associated packet payloads, and additional contextual features to address significant limitations in current publicly available datasets. These limitations include inconsistencies in feature sets, the absence of payload-level data, and the lack of time-window-based contextual information. By standardizing the feature space across different datasets and incorporating both flow and payload data, our approach significantly enhances the ability to detect sophisticated attacks, such as those relying on payload characteristics and time-sensitive behaviors.

Our experimental analysis highlights the value of these contextual features and payload data in improving the performance of machine learning-based NIDS. Specifically, our results demonstrate that models trained with the unified dataset and enriched features outperform those relying solely on flow-level data. Furthermore, the flexibility of the tool we developed enables users to customize the dataset according to their specific research needs, including adjusting the sampling strategy and contextual feature settings, making it adaptable for a wide range of scenarios.

The final labeled UM-NIDS dataset provides a robust foundation for cross-dataset validation and the development of more generalized models, addressing a critical gap in the field of network intrusion detection. This contribution significantly advances the ability of researchers to develop and evaluate NIDS models using a comprehensive, multimodal dataset that supports more accurate and versatile threat detection.

SOURCE CODE AND SCRIPTS

The UM-NIDS Tool is designed to facilitate the generation of the unified multimodal NIDS dataset. This tool enables users to standardize datasets, ensuring consistent feature space, payload content, and contextual attributes across multiple NIDS datasets. Additionally, it allows for the extension of the dataset by processing additional raw PCAP files into the same unified format. To extend the dataset or standardize another dataset, the basic requirements include the PCAP files and their corresponding pre-labeled CSV files. These two elements are essential for converting the data into this unified format.

An additional feature in the repository provides flexibility in the labeling process. If the labeled CSV files are not available, the tool can also label the PCAP files using the MAC addresses of attackers, if such metadata are available. This provision allows users to perform labeling and conversion into the unified dataset format without needing pre-labeled CSVs. Examples demonstrating both approaches, using pre-labeled CSVs and MAC addresses for labeling, are included in our GitHub repository. The source code and scripts for this tool are also available on GitHub, allowing users to easily extend and modify the dataset.

<https://github.com/SyedWaliAbbas/UM-NIDS-Tool>

ACKNOWLEDGMENT

S.W. contributed to the methodology, data curation, formal analysis, and experimentation and led the writing of the original draft. Y.A.F. contributed to the methodology, visualization, investigation, and formal analysis. I.K. provided supervision and resources. N.D.B. contributed to conceptualization, methodology, and project administration. All authors reviewed and approved the final manuscript.

The views and conclusions expressed in this article are those of the authors and do not reflect the official policy or position of the U.S. Military Academy, U.S. Army, U.S. Department of Defense, or U.S. Government.

The authors have declared no conflicts of interest.

REFERENCES

- [1] M. Ozkan-Okay, R. Samet, Ö. Aslan, and D. Gupta, "A comprehensive systematic literature review on intrusion detection systems," *IEEE Access*, vol. 9, pp. 157727–157760, 2021.
- [2] Y. A. Farrukh, S. Wali, I. Khan, and N. D. Bastian, "SeNet-I: An approach for detecting network intrusions through serialized network traffic images," *Eng. Appl. Artif. Intell.*, vol. 126, 2023, Art. no. 107169.
- [3] S. Gamage and J. Samarabandu, "Deep learning methods in network intrusion detection: A survey and an objective comparison," *J. Netw. Comput. Appl.*, vol. 169, 2020, Art. no. 102767.
- [4] I. Ghazali, M. F. Asy'ari, S. Triarjo, H. M. Ramadhani, H. Studiawan, and A. M. Shiddiqi, "A novel SQL injection detection using Bi-LSTM and TF-IDF," in *Proc. 7th Int. Conf. Inf. Netw. Technol. (ICINT)*, Piscataway, NJ, USA: IEEE, 2022, pp. 16–22.
- [5] R. Pallam, S. P. Konda, L. Manthripragada, and R. A. Noone, "Detection of web attacks using ensemble learning," *Learning*, vol. 3, no. 4, p. 5, 2021.
- [6] Y. A. Farrukh, S. Wali, I. Khan, and N. D. Bastian, "XG-NID: Dual-modality network intrusion detection using a heterogeneous graph neural network and large language model," 2024, *arXiv:2408.16021*.
- [7] A. Thakkar and R. Lohiya, "A review of the advancement in intrusion detection datasets," *Procedia Comput. Sci.*, vol. 167, pp. 636–645, 2020.
- [8] M. Sarhan, S. Layeghy, and M. Portmann, "Towards a standard feature set for network intrusion detection system datasets," *Mob. Netw. Appl.*, vol. 27, pp. 1–14, 2022.
- [9] M. Ghurab, G. Gaphari, F. Alshami, R. Alshamy, and S. Othman, "A detailed analysis of benchmark datasets for network intrusion detection system," *Asian J. Res. Comput. Sci.*, vol. 7, no. 4, pp. 14–33, 2021.
- [10] Y. A. Farrukh, I. Khan, S. Wali, D. Bierbrauer, J. A. Pavlik, and N. D. Bastian, "Payload-Byte: A tool for extracting and labeling packet capture files of modern network intrusion detection datasets," in *Proc. IEEE/ACM Int. Conf. Big Data Comput., Appl. Technol. (BDCAT)*, Piscataway, NJ, USA: IEEE, 2022, pp. 58–67.
- [11] I. Sharafaldin et al., "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. ICISSp*, vol. 1, 2018, pp. 108–116.
- [12] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy," in *Proc. Int. Carnahan Conf. Secur. Technol. (ICCST)*, Piscataway, NJ, USA: IEEE, 2019, pp. 1–8.
- [13] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf. (MilCIS)*, Piscataway, NJ, USA: IEEE, 2015, pp. 1–6.
- [14] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, 2023, Art. no. 5941.
- [15] Z. Aouini and A. Pekar, "NFStream: A flexible network data analysis framework," *Comput. Netw.*, vol. 204, 2022, Art. no. 108719.